

به نام خدا



گزارش نهایی پروژه‌ی پیام‌رسان Discord

تحلیل نحوه‌ی برآورده‌سازی نیازمندی‌های کارکردی و غیرکارکردی

درس تحلیل و طراحی سیستم‌ها

نیم‌سال ۱۴۰۴-۱۴۰۵

گروه ۱

اعضای گروه: عرفان تیموری، امیرمهدی طهماسبی، امیرمسعود ابراهیمی، پوریا رحمانی، امیرحسین یگانه‌دوست، مهدی شکوفی

استاد و دستیار آموزشی: دکتر فضلی - آقای ابراهیمی - آقای جعفری‌پور

فهرست مطالب

۱. مقدمه و هدف گزارش.....	۳
۲. معماری و فناوری‌های استفاده شده.....	۳
۳. نیازمندی‌های کارکردی.....	۴
۳-۱. Epic اول: هویت، پروفایل و جست‌وجوی کاربران.....	۴
۳-۲. Epic دوم: مدیریت مکالمه و پیام.....	۵
۳-۳. Epic سوم: مدیریت گروه‌های خصوصی.....	۷
۳-۴. Epic چهارم: کانال، Topic و کنترل دسترسی.....	۹
۳-۵. Epic پنجم: قابلیت‌های زنده، اعلان و پیام زمان‌بندی شده.....	۱۲
۴. نیازمندی‌های غیرعملکردی.....	۱۳
۵. انطباق محصول با طراحی و معماری.....	۱۵

۱. مقدمه و هدف گزارش

در این گزارش، ارتباط بین نیازمندی‌های استخراج شده در تحلیل سامانه و طراحی پروژه و همچنین تطابق طراحی با تحلیل سامانه توضیح داده شده است. نیازمندی‌های عملکردی بر پایه **User story** های نهایی دسته بندی شده اند و برای هر مورد، توضیحات مختصری در مورد نحوه پاسخ دهی به آن **User story** در سطح رابط کاربری، **API**، منطق دامنه و زیرساخت بیان شده است.

سامانه نهایی یک پیام رسان وب مشابه **Discord** است. این سامانه، احراز هویت، پروفایل کاربری، چت شخصی، گروه خصوصی، کانال و **Topic**، نقش های مدیریتی، پیام متنی و رسانه ای، جستجو کاربران و کانال ها، ارتباط بلادرنگ، اعلان ها و پیام های زمان بندی شده را به صورت هماهنگ پیاده سازی کرده است.

۲. معماری و فناوری‌های استفاده شده

محصول با معماری وب چندلایه و **Backend** مازولار توسعه یافته است. رابط کاربری **React/TypeScript** از طریق **REST API** و **WebSocket** با بک‌اند **Django** ارتباط دارد. منطق دامنه در برنامه‌های **accounts**، **chats**، **messaging** و **core** تفکیک شده و داده‌های اصلی در **PostgreSQL** نگهداری می‌شوند.

بخش	فناوری مورد استفاده	نحوه پاسخ به محصول
رابط کاربری	React ، TypeScript و Vite	صفحات احراز هویت، پروفایل، جست‌وجو، چت، گروه، کانال، اعلان و پیام‌های زمان‌بندی شده را ارائه می‌کند.
API و منطق دامنه	Django REST Framework و Django	اعتبارسنجی، مدیریت نشست، عملیات پیام‌رسانی و کنترل دسترسی مبتنی بر مالکیت و عضویت را اجرا می‌کند.
ارتباط بلادرنگ	Django Channels ، ASGI و Redis	رویدادهای ایجاد، ویرایش و حذف پیام، وضعیت خواندن و اعلان شخصی را بدون refresh منتقل می‌کند.
پردازش پس‌زمینه	Celery و Redis	پیام‌های زمان‌بندی شده را مستقل از آنلاین بودن فرستنده در زمان مقرر تحویل می‌دهد.
داده و فایل	PostgreSQL و MinIO/S3	داده‌های رابطه‌ای و وضعیت‌ها را پایدار نگه می‌دارد و فایل‌های عمومی و خصوصی را در مخازن جدا ذخیره می‌کند.

بخش	فناوری مورد استفاده	نحوه پاسخ به محصول
استقرار	Docker Compose	سرویس‌های frontend، backend، PostgreSQL، Redis، MinIO و Celery را با پیکربندی محیطی یکپارچه می‌کند.

۳. نیازمندی‌های کارکردی

۳-۱. Epic اول: هویت، پروفایل و جست‌وجوی کاربران

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-01	ثبت‌نام کاربر جدید	فرم ثبت‌نام React اطلاعات نام، نام خانوادگی، شماره تلفن، جنسیت و رمز عبور را دریافت می‌کند. RegisterSerializer اعتبارسنجی رمز را انجام می‌دهد و مدل User با کلید اصلی شماره تلفن از ایجاد حساب تکراری جلوگیری می‌کند؛ حساب بدون OTP یا تأیید ایمیل ساخته می‌شود.
US-02	ورود به حساب کاربری	LoginSerializer شماره تلفن و رمز عبور را اعتبارسنجی می‌کند و در صورت موفقیت، نشست اختصاصی برنامه ایجاد می‌شود. فرم ورود نیز خطاهای نامعتبر بودن اطلاعات را به‌صورت روشن نمایش می‌دهد.
US-03	خروج از حساب کاربری	عملیات خروج کلید نشست برنامه را در سرور حذف می‌کند و AuthContext وضعیت محلی کاربر را پاک می‌سازد. محافظ مسیرهای React پس از خروج، صفحات خصوصی را فقط پس از ورود مجدد در دسترس قرار می‌دهد.
US-04	مشاهده پروفایل شخصی	مسیر GET /api/auth/me اطلاعات نام، شماره تلفن، جنسیت، bio، آواتار و tag را برمی‌گرداند و صفحه تنظیمات آن‌ها را نمایش می‌دهد. رمز عبور و hash آن در Serializer قرار نگرفته و هرگز به کلاینت ارسال نمی‌شود.
US-05	ویرایش اطلاعات پروفایل	مسیر PATCH /api/auth/me و فرم SettingsPage ویرایش نام، جنسیت، bio، tag و تنظیمات حریم خصوصی را پشتیبانی می‌کنند. برای آواتار از درخواست multipart استفاده شده و پس از ذخیره، نمایه به‌روز در رابط کاربری نمایش داده می‌شود.

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-06	مشاهده پروفایل کاربر دیگر	PublicUserSerializer فقط اطلاعات عمومی را ارائه می‌دهد. UserProfileDialog با انتخاب نام یا آواتار از داخل گفتگو باز می‌شود و بدون ترک صفحه، نام، تلفن، bio، آواتار و tag کاربر را نمایش می‌دهد.
US-07	جست‌وجوی کاربران	UserSearchView جست‌وجوی جزئی روی نام، نام خانوادگی و شماره تلفن را انجام می‌دهد. نتایج در صفحه جست‌وجو و MemberAdder استفاده شده‌اند تا چت شخصی آغاز شود یا کاربر به گروه و کانال افزوده شود؛ لینک‌های دعوت نیز از پنل اطلاعات فضا قابل کپی هستند.

۳-۲. Epic دوم: مدیریت مکالمه و پیام

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-08	شروع چت شخصی	سرویس get_or_create_direct_chat با ساخت direct_key قطعی از شناسه مرتب‌شده دو کاربر، تنها یک گفت‌وگوی شخصی برای هر زوج ایجاد می‌کند. عضویت دقیقاً برای همان دو کاربر ثبت می‌شود و کنترل دسترسی، مشاهده پیام‌ها را به آن‌ها محدود می‌کند.
US-09	ارسال پیام متنی	MessageComposer پیام را به API گفت‌وگوی انتخاب‌شده می‌فرستد و create_text_message پس از کنترل عضویت و مجوز ارسال، متن غیرخالی را همراه فرستنده، مقصد و زمان در PostgreSQL ذخیره می‌کند. این مسیر برای چت شخصی، گروه و Topic مشترک است.
US-10	مشاهده تاریخچه پیام‌ها	MessageHistoryView پیام‌های حذف‌نشده را به ترتیب زمانی و با صفحه‌بندی مبتنی بر cursor برمی‌گرداند. رابط ChatPage بارگذاری اولیه، دریافت پیام‌های قدیمی‌تر و جدیدتر و قرارگیری روی نخستین پیام خوانده‌نشده را مدیریت می‌کند.
US-11	ارسال و دریافت رسانه	multipart ارسال و متادیتای آن در مدل File ثبت می‌شود. محتوای فایل در فضای خصوصی MinIO/S3 نگهداری شده و AttachmentDownloadView تنها پس از کنترل عضویت در همان گفت‌وگو، فایل را به‌صورت دانلودی ارائه می‌کند.

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-12	ویرایش متن پیام	مسیر PATCH پیام و سرویس <code>edit_message</code> فقط به فرستنده اجازه ویرایش می‌دهند. متن جدید اعتبارسنجی و <code>is_edited</code> فعال می‌شود؛ سپس نسخه جدید از مسیر <code>WebSocket</code> برای کاربران حاضر در گفتگو پخش و با برچسب <code>edited</code> نمایش داده می‌شود.
US-13	ویرایش فایل ضمیمه پیام	همان <code>API</code> ویرایش امکان جایگزینی یا حذف فایل ضمیمه را دارد. فایل جدید ابتدا در فضای خصوصی ذخیره می‌شود، اتصال پیام در تراکنش تغییر می‌کند و فایل قبلی پس از موفقیت عملیات پاک‌سازی می‌شود؛ این دسترسی فقط برای فرستنده است.
US-14	حذف پیام توسط فرستنده	<code>DELETE</code> پیام از سرویس <code>delete_message</code> استفاده می‌کند. رکورد با <code>is_deleted</code> به‌صورت نرم حذف، فایل ضمیمه از دسترس خارج و رویداد <code>message.deleted</code> برای حذف فوری پیام از رابط تمام کاربران گفتگو منتشر می‌شود.
US-15	حذف پیام دیگران توسط مدیر	تابع <code>can_delete_message</code> علاوه بر فرستنده، <code>Owner</code> گروه و <code>Admin</code> یا <code>Owner</code> کانال را برای حذف پیام‌های فضای تحت مدیریت مجاز می‌داند. <code>API</code> و رابط کاربری همین مجوز را اعمال کرده و اعضای عادی امکان حذف پیام دیگران را ندارند.
US-16	جست‌وجوی پیام‌ها در مکالمه	<code>MessageSearchView</code> عبارت را فقط روی پیام‌های قابل مشاهده و حذف‌نشده همان <code>chat</code> جست‌وجو می‌کند. دسترسی به <code>chat</code> پیش از جست‌وجو بررسی می‌شود و جست‌وجوی زنده در <code>ChatPage</code> پس از مکث کوتاه تایپ اجرا می‌گردد.
US-17	نمایش نتایج جست‌وجوی پیام	هر نتیجه شامل پیش‌نمایش متن، فرستنده، زمان و وضعیت ویرایش است. <code>MessageContextView</code> یک پنجره زمانی پیرامون نتیجه می‌سازد و رابط کاربری با انتخاب نتیجه، همان بخش تاریخچه را بارگذاری، پیام را برجسته و حالت بدون نتیجه را نمایش می‌دهد.

۳-۳. Epic سوم: مدیریت گروه‌های خصوصی

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-18	ایجاد گروه خصوصی	کامپوننت ری‌اکت <code>CreateGroupDialog</code> که در پوشه <code>frontend</code> های <code>component</code> در پوشه <code>group</code> قرار دارد، مشخصات گروه را دریافت و <code>API</code> گروه را فراخوانی می‌کند. سرویس <code>create_group</code> که در <code>backend/chats/services.py</code> قابل مشاهده است، گروه را با سطح دسترسی خصوصی می‌سازد و یک لینک دعوت یکتا برای آن تشکیل می‌دهد. همچنین سازنده گروه، به عنوان <code>owner</code> به اعضای گروه اضافه می‌شود.
US-19	مشاهده گروه در فهرست مکالمات	گروه ها با کامپوننت <code>ConversationSidebar</code> که در پوشه <code>component</code> های <code>frontend</code> در پوشه <code>chat</code> قرار دارد، نشان داده می‌شوند. همچنین با استفاده از <code>GroupMembership</code> در <code>backend/chats/models.py</code> اطمینان حاصل می‌شود که گروه فقط به اعضایش نشان داده شود. می‌توان در فهرست مکالمات، نام، <code>bio</code> ، آواتار، وضعیت پیام‌های خوانده نشده و ... را دید.
US-20	افزودن عضو به گروه	<code>Owner</code> از پنل اطلاعات گروه کاربران را با نام یا شماره تلفن جست‌وجو می‌کند. سرویس <code>add_group_member</code> (در پوشه <code>chats</code>)، بررسی می‌کند که کاربری که درخواست اضافه کردن به گروه را می‌دهد واقعا <code>owner</code> باشد و کاربر هدف هم در تنظیماتش اضافه شدن به گروه توسط دیگران را مسدود نکرده باشد و در صورت برقرار بودن همه شرایط، یک <code>GroupMembership</code> ایجاد می‌کند.
US-21	حذف عضو از گروه	عملیات حذف عضو تنها توسط <code>Owner</code> قابل انجام است و اگر کاربری از گروه حذف شود، رکورد <code>GroupMembership</code> متناظر او حذف می‌شود. کنترل دسترسی <code>REST</code> و <code>WebSocket</code> عضویت در گروه را به طور مداوم از طریق <code>can_access_chat</code> در <code>backend/chats/permissions.py</code> بررسی می‌کند و به محض حذف، دسترسی به پیام‌های جدید از کاربر حذف شده برداشته می‌شود.

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-22	تنظیم مجوز اضافه‌شدن به گروه	کاربران در مدل خود یک فیلد <code>can_be_added_to_group</code> دارند که قابل <code>set</code> کردن یا برداشتن است. هنگامی که <code>owner</code> می‌خواهد کسی را به گروه اضافه کند، سرویس <code>add_group_member</code> که توضیحش پیشتر آمد صدا زده می‌شود و این سرویس مقدار این فیلد را چک می‌کند و اگر <code>false</code> باشد خطا بر می‌گرداند. توجه کنید در هر صورت کاربر می‌تواند از طریق لینک دعوت عضو گروه بشود.
US-23	ویرایش اطلاعات گروه	کامپوننت <code>GroupInfoDialog</code> و مسیر <code>PATCH /api/groups/{id}/</code> (می‌توانید برای بررسی بیشتر کلاس <code>GroupDetailView</code> در <code>backend/chats/views.py</code> را مشاهده کنید) امکان تغییر نام، <code>bio</code> ، <code>tag</code> ، سطح دسترسی و تنظیم رسانه را فراهم می‌کنند. سرویس <code>update_group</code> پیش از اعمال تغییرات، <code>Owner</code> بودن کاربر و معتبر بودن فیلدها را کنترل می‌کند.
US-24	ویرایش آواتار گروه	آواتار در مدل <code>Group</code> ، یک <code>Django ImageField</code> مجزا است. برای هندل کردن آپلود تصاویر، <code>update</code> و <code>create</code> در <code>backend/chats/views.py</code> از <code>multipart request</code> پشتیبانی می‌کنند. بعد از <code>validation</code> ، آواتار جدید ذخیره می‌شود و از طریق <code>url</code> اش قابل دسترسی است.
US-25	حذف گروه	گزینه حذف فقط برای <code>Owner</code> در <code>GroupInfoDialog</code> نمایش داده می‌شود و <code>ConfirmDialog</code> تأیید نهایی می‌گیرد. سرویس <code>delete_group</code> ، پرچم <code>is_deleted</code> متناظر آن <code>Group</code> (توجه کنید <code>Group</code> از <code>Chat</code> ارث بری می‌کند و بنابراین <code>is_deleted</code> دارد) را ست می‌کند تا حذف به صورت <code>Soft_delete</code> انجام شود. با اینکار تاریخچه گروه در دیتابیس باقی می‌ماند اما از <code>user interface</code> پاک می‌شود.
US-26	ترک گروه	عضویت گروه به‌صورت رکورد مستقل <code>GroupMembership</code> مدیریت می‌شود. خروج یک عضو از یک گروه با حذف رکورد <code>GroupMembership</code> متناظر او صورت می‌گیرد. بعد از آن، <code>can_access_chat</code> که پیشتر هم از آن نام بردیم، مانع دسترسی فرد خارج شده به گروه می‌شود. <code>owner</code> گروه تنها گزینه حذف گروه را دارد، یعنی خروج <code>owner</code> ، منجر به حذف گروه برای همه اعضا می‌شود.

۳-۴. Epic چهارم: کانال، Topic و کنترل دسترسی

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-27	ایجاد کانال	کامپوننت فرانت‌اند <code>CreateChannelDialog</code> (واقع در پوشه <code>channel</code>) نام، <code>bio</code> ، <code>tag</code> ، آواتار و نوع <code>public/private</code> را جمع‌آوری و به <code>API</code> ارسال می‌کند. سرویس <code>create_channel</code> کانال را در یک <code>transcation</code> دیتابیس می‌سازد، سازنده را <code>Owner</code> قرار می‌دهد و یک رکورد <code>ChannelMembership</code> از نوع <code>admin</code> ثبت می‌کند.
US-28	هویت یکتای کانال عمومی	نام کانال‌های فعال با شرط <code>UniqueConstraint</code> در دیتابیس (بدون حساسیت به بزرگی و کوچکی حروف) یکتا شده است و سرویس ساخت و ویرایش نیز پیش از ذخیره کردن نام کانال، این ویژگی را کنترل می‌کنند. علاوه بر آن، هر کانال یک <code>token</code> دعوت یکتای غیرقابل حدس دارد.
US-29	جستجوی کانال‌های عمومی	کلاس <code>ChannelDirectoryView</code> در <code>backend/chats/views.py</code> که از <code>APIView</code> <code>rest_framework</code> ارث‌بری می‌کند، جستجو را روی نام و <code>bio</code> انجام می‌دهد اما <code>QuerySet</code> را از ابتدا به <code>access_level=public</code> محدود می‌کند. با این کار، تنها کانال‌های عمومی در نتایج جستجو ظاهر می‌شوند. در فرانت‌اند نیز، کامپوننت ری‌اکت <code>searchResultsPage.tsx</code> در پوشه <code>pages</code> ، اطلاعات گروه‌های <code>match</code> شده در جستجو، شامل تعداد اعضا و گزینه <code>join</code> را نشان می‌دهد.
US-30	ایجاد لینک دعوت کانال	در زمان تولید کانال‌ها، یک <code>token</code> یکتا برای آن‌ها تولید می‌شود که <code>Owner</code> یا <code>Admin</code> می‌تواند آن را از کامپوننت <code>ChannelInfoDialog</code> مشاهده، کپی یا <code>rotate</code> کند. <code>Rotate</code> کردن توسط سرویس رکاند <code>rotate_channel_invite</code> صورت می‌گیرد که توکن جدیدی می‌سازد و توکن قبلی را باطل می‌کند.
US-31	عضویت در کانال با لینک دعوت	<code>JoinChannelPage</code> ابتدا یک <code>preview</code> از اطلاعات کانال را برای تأیید کاربر قبل از شروع فرایند عضویت نشان می‌دهد، سپس در <code>backend</code> ، <code>ChannelInviteJoinView</code> توکن را <code>verify</code> می‌کند و در صورت درست بودن <code>token</code> ، سرویس بک‌اند <code>join_channel_via_token</code> را صدا می‌زند تا یک رکورد <code>ChannelMembership</code> بسازد.

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-32	حذف عضو از کانال	Owner و Admin از پنل اعضا امکان حذف Member را دارند. سرویس بک‌اند <code>remove_channel_member</code> ، وظیفه بررسی معتبر بودن عملیات حذف را بررسی کند. (کسی حق ندارد owner را حذف کند و یک admin نمی‌تواند admin دیگر را حذف کند) پس از حذف، کاربر دسترسی به همه topic ها را از دست می‌دهد.
US-33	ایجاد Topic	کامپوننت فرانت‌اند <code>TopicDialog</code> و ویو <code>TopicListCreateView</code> وظیفه ایجاد Topic را درون کانال را دارند. سرویس <code>create_topic</code> بررسی می‌کند که کسی که درخواست ساخت topic را داده، حتماً admin یا owner باشد. وجود foreign key از کانال در مدل <code>topic</code> ، تضمین می‌کند هر topic تنها متعلق به یک کانال باشد.
US-34	مشاهده Topic های کانال	صفحه فرانت‌اند <code>ChannelPage</code> و ویو بک‌اند <code>TopicListCreateView</code> فهرست Topic های فعال را مرتب‌سازی کرده و نمایش می‌دهند. تابع <code>can_discover_channel</code> در <code>backend/chats/permissions.py</code> تضمین می‌کند تاپیک‌ها و جزئیات کانال‌های خصوصی از دید افرادی که عضو آن کانال‌ها نیستند پنهان بماند. تاپیک‌های یک کانال، در صفحه اصلی کانال برای کاربران قابل مشاهده‌اند.
US-35	ارسال پیام متنی در Topic	Topic از Chat ارث می‌برد و از همان مسیر استاندارد ارسال پیام استفاده می‌کند. <code>can_access_chat</code> عضویت در کانال و <code>can_send_to_chat</code> وضعیت اجازه ارسال اعضا را بررسی می‌کند؛ پیام همراه شناسه همان Topic ذخیره و بلادرنگ منتشر می‌شود.
US-36	ویرایش اطلاعات کانال	<code>ChannelInfoDialog</code> و مسیر PATCH کانال ویرایش <code>bio</code> ، <code>tag</code> ، آواتار، نوع دسترسی و تنظیم رسانه را ارائه می‌کنند. <code>update_channel</code> عملیات را به نقش‌های مدیریتی محدود و فیلدهای ورودی را پیش از ذخیره اعتبارسنجی می‌کند.
US-37	تغییر نام کانال	نام کانال از فرم مدیریت و API ویرایش می‌شود. سرویس، نام خالی یا تکراری را رد می‌کند و <code>UniqueConstraint</code> پایگاه داده نیز یکتایی بدون حساسیت به حروف را تضمین می‌نماید؛ کنترل نقش پیش از انجام تغییر اعمال می‌شود.

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-38	حذف کانال	حذف کانال فقط برای Owner در API و رابط مدیریت در دسترس است و پس از ConfirmDialog انجام می‌شود. delete_channel کانال و Topic های وابسته را در یک تراکنش soft delete می‌کند تا از دید و دسترسی اعضا خارج شوند.
US-39	اختصاص نقش Admin	Owner از فهرست اعضای ChannelInfoDialog نقش عضو را تغییر می‌دهد. ChannelMemberDetailView و set_channel_admin فقط درخواست Owner را می‌پذیرند و با فعال کردن is_admin، دسترسی‌های مدیریتی بلافاصله در پاسخ API و رابط اعمال می‌شوند.
US-40	پس گرفتن نقش Admin	همان endpoint عضویت با is_admin=false کاربر را به Member عادی تبدیل می‌کند. تغییر نقش تنها توسط Owner مجاز است و بررسی‌های بعدی مجوز مستقیماً مقدار جدید عضویت را می‌خوانند.
US-41	اعمال نقش‌های کانال	توابع متمرکز is_channel_owner، is_channel_admin و is_channel_member سیاست نقش‌ها را در تمام سرویس‌ها اعمال می‌کنند. حذف کانال و مدیریت نقش در اختیار Owner، مدیریت Topic و اعضا و دعوت در اختیار Owner/Admin و ارسال عادی در اختیار Member قرار گرفته است.
US-42	تنظیم دسترسی ارسال رسانه در کانال	فیلد allow_media روی Channel قرار دارد و Owner/Admin آن را از ChannelInfoDialog فعال یا غیرفعال می‌کنند. can_send_media_to_chat این تنظیم را فقط برای Topic های همان کانال اعمال می‌کند و چت شخصی و گروه سیاست مستقل خود را دارند.
US-43	جلوگیری از ارسال رسانه توسط Member محدودشده	رابط ChatPage براساس allow_media و نقش کاربر گزینه پیوست را غیرفعال می‌کند و یک‌اند نیز مستقل از رابط، آپلود غیرمجاز را با پاسخ دسترسی رد می‌نماید. Owner و Admin از محدودیت Member ها مستثنا هستند.

۵-۳. Epic: قابلیت‌های زنده، اعلان و پیام زمان‌بندی‌شده

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-44	دریافت زنده پیام‌های جدید	Django Channels برای هر chat یک گروه WebSocket می‌سازد و فقط کاربران احرازشده و دارای دسترسی را می‌پذیرد. رویداد پیام پس از commit پایگاه داده منتشر می‌شود؛ ChatPage بدون refresh پیام را ادغام و در قطع اتصال با backoff متصل و تاریخچه از دست‌رفته را بازیابی می‌کند.
US-45	نمایش زنده پیام ویرایش‌شده	پس از edit_message، وضعیت canonical پیام از همان کانال بلادرنگ منتشر می‌شود. کلاینت با شناسه پیام نسخه قبلی را جایگزین می‌کند و is_edited را با برچسب edited نمایش می‌دهد؛ ارسال رویداد نیز به اعضای مجاز همان chat محدود است.
US-46	اعلان پیام برای کاربر آنلاین	یک WebSocket شخصی مستقل برای هر کاربر همه مکالمات او را پوشش می‌دهد. پیام جدید علاوه بر افزایش unread badge، در صورت mute نبودن و بازنبودن همان chat به شکل toast شامل فرستنده، مقصد و پیش‌نمایش نشان داده می‌شود و کاربر می‌تواند مستقیماً به گفتگو برود.
US-47	زمان‌بندی پیام متنی	ScheduleMessageDialog مقصد، متن، تاریخ و ساعت محلی را دریافت و زمان را به ISO ارسال می‌کند. سرویس زمان گذشته یا متن خالی را رد کرده و datetime آگاه از منطقه زمانی را به UTC تبدیل و با وضعیت pending ذخیره می‌نماید.
US-48	ارسال خودکار پیام زمان‌بندی‌شده	Celery پیام را با ETA دقیق تحویل می‌دهد و یک sweeper دوره‌ای موارد دوردست یا جامانده را دوباره صف‌بندی می‌کند. هنگام ارسال، مجوز فرستنده مجدداً بررسی و با قفل ردیف از تحویل تکراری جلوگیری می‌شود؛ وضعیت در پایان sent یا failed خواهد بود.
US-49	مشاهده پیام‌های زمان‌بندی‌شده	ScheduledMessageListView تنها پیام‌های متعلق به کاربر جاری را برمی‌گرداند. ScheduledMessagesPage مقصد، پیش‌نمایش متن، زمان محلی/نسبی و وضعیت هر مورد را به‌صورت گروه‌بندی‌شده نمایش می‌دهد.

شناسه	نیازمندی کارکردی	نحوه پاسخ محصول و شواهد پیاده‌سازی
US-50	حذف پیام زمان‌بندی‌شده	کاربر می‌تواند پیام <code>pending</code> خود را از صفحه زمان‌بندی لغو کند. <code>cancel_scheduled_message</code> مالکیت و وضعیت را زیر قفل تراکنشی بررسی و وضعیت را <code>cancelled</code> می‌کند؛ <code>worker</code> نیز فقط رکورد <code>pending</code> را تحویل می‌دهد، بنابراین مورد لغوشده ارسال نمی‌شود.

۴. نیازمندی‌های غیرعملکردی

نیازمندی‌های غیرعملکردی زیر از انتظارات کیفی یک سامانه پیام‌رسان و با توجه به محصول نهایی استخراج شده است. هر ردیف نشان می‌دهد راهکار فنی چگونه آن ویژگی کیفی را تأمین می‌کند.

شناسه	نیازمندی غیرکارکردی	نحوه تحقق در محصول
NFR-01	امنیت احراز هویت	رمزها با سازوکار استاندارد <code>Django hash</code> و با <code>Password Validator</code> ها بررسی می‌شوند. احراز هویت برنامه مبتنی بر نشست است، درخواست‌های تغییردهنده <code>CSRF</code> دارند و <code>CORS</code> فقط برای <code>origin</code> های مجاز همراه <code>credential</code> فعال شده است.
NFR-02	مجوزدهی و حریم خصوصی	مجوزها در توابع دامنه متمرکز شده و در <code>API</code> ، دانلود فایل و اتصال <code>WebSocket</code> اعمال می‌شوند. عضویت هنگام هر رویداد <code>blادرنگ</code> دوباره بررسی می‌شود تا حذف عضو فوراً اثر کند و اطلاعات خصوصی فقط به کاربران مجاز برسد.
NFR-03	محرمانگی فایل‌ها	آواتارها و فایل‌های پیام در <code>bucket</code> های جدا نگهداری می‌شوند. فایل پیام <code>URL</code> عمومی ندارد و فقط از <code>AttachmentDownloadView</code> پس از کنترل دسترسی <code>stream</code> می‌شود؛ نام فایل پاک‌سازی و مسیر ذخیره‌سازی نیز با شناسه تصادفی ساخته می‌شود.
NFR-04	کارایی و زمان پاسخ	تاریخچه با <code>cursor</code> و محدودیت اندازه صفحه بارگذاری می‌شود، نتایج جست‌وجو محدود هستند و <code>QuerySet</code> ها از <code>select_related</code> و <code>prefetch_related</code> استفاده می‌کنند. وضعیت خواندن به‌صورت <code>high-water mark</code> ذخیره شده و از ایجاد رکورد برای هر پیام و هر خواننده جلوگیری می‌کند.

شناسه	نیازمندی غیرکارکردی	نحوه تحقق در محصول
NFR-05	مقیاس‌پذیری	PostgreSQL برای داده پایدار، Redis برای Channel Layer و Celery broker برای workerهای مستقل و MinIO/S3 برای Object Storage استفاده شده است. جداسازی سرویس‌ها امکان افزایش نمونه‌های worker، backend و فضای ذخیره‌سازی را بدون تغییر منطق محصول فراهم می‌کند.
NFR-06	قابلیت اطمینان و تحمل خطا	عملیات چندمرحله‌ای در transaction.atomic انجام و رویدادها با on_commit منتشر می‌شوند. Celery retry با backoff و jitter دارد و ترکیب ETA و sweeper پیام‌های جامانده را بازیابی می‌کند؛ قفل سطری نیز تحویل دقیقاً یک‌باره پیام زمان‌بندی‌شده را تضمین می‌نماید.
NFR-07	یکپارچگی و سازگاری داده	قیود یکتایی برای عضویت‌ها، direct_key، نام کانال، mute و read state در پایگاه داده تعریف شده‌اند. soft delete تاریخچه را حفظ می‌کند، کلیدهای خارجی روابط را کنترل می‌کنند و تمام زمان‌های زمان‌بندی‌شده به UTC نرمال می‌شوند.
NFR-08	نگهداشت‌پذیری و توسعه‌پذیری	بک‌اند به ماژول‌های accounts، chats، messaging، core و لایه‌های model، serializer، service و view تفکیک شده است. فرانت‌اند TypeScript نیز API‌ها، context‌ها، hook‌ها و کامپوننت‌های قابل استفاده مجدد را جدا کرده و تغییر هر قابلیت را محلی نگه می‌دارد.
NFR-09	قابلیت استقرار و حمل‌پذیری	Docker Compose سرویس‌های frontend، backend، PostgreSQL، Redis، MinIO و Celery را با تنظیمات محیطی هماهنگ اجرا می‌کند. وابستگی‌ها در فایل‌های lock و requirements مشخص‌اند و تنظیمات حساس از environment خوانده می‌شوند.
NFR-10	کاربردپذیری و دسترس‌پذیری رابط	رابط واکنش‌گرا دارای وضعیت‌های loading، empty و error، تأیید عملیات مخرب، شمارنده خوانده‌نشده، اعلان toast، برجسته‌سازی جست‌وجو و بازخورد اتصال است. aria-label، role‌های وضعیت و کنترل‌های صفحه‌کلید برای تعاملات اصلی به‌کار رفته‌اند.

۵. انطباق محصول با طراحی و معماری

هسته محصول یک بک‌اند ماژولار است و قابلیت‌هایی که واقعاً به زیرساخت جدا نیاز دارند (ارتباط زنده، صف وظایف و ذخیره فایل) به Celery، Redis و MinIO واگذار شده‌اند. این تصمیم پیچیدگی توسعه را کنترل کرده و درعین‌حال مسیر مقیاس‌پذیری را باز نگه داشته است.

جنبه طراحی	انتظار معماری	تحقق در محصول نهایی
معماری کلان	کلاینت وب و بک‌اند ماژولار در یک monorepo	React به‌عنوان SPA و Django به‌عنوان API/ASGI مرز روشنی دارند؛ منطق دامنه در ماژول‌های messaging، chats، accounts و core تفکیک شده است.
مدل دامنه	تفکیک کاربر، مکالمه، گروه، کانال، Topic، عضویت، نقش و پیام	مدل‌های رابطه‌ای و قیود پایگاه داده مستقیماً مفاهیم سند نیازمندی را نمایش می‌دهند و از تکرار عضویت، چت شخصی و نام کانال جلوگیری می‌کنند.
ارتباط همگام	عملیات CRUD و بازیابی داده از طریق API	REST API برای احراز هویت، پروفایل، گروه، کانال، Topic، پیام، فایل و زمان‌بندی استفاده شده و Serializerها مرز اعتبارسنجی ورودی و خروجی هستند.
ارتباط ناهمگام	تحويل فوری رویداد و اجرای کارهای آینده	WebSocket برای پیام و اعلان زنده و Celery برای زمان‌بندی به‌کار رفته است؛ Redis زیرساخت مشترک Channel Layer و صف وظایف را فراهم می‌کند.
ذخیره‌سازی	پایداری داده و تفکیک فایل عمومی و خصوصی	PostgreSQL داده‌های ساختاریافته را نگهداری می‌کند و MinIO/S3 آواتارها و ضمیمه‌های خصوصی را در bucketهای جدا قرار می‌دهد.
امنیت معماری	احراز هویت، مجوز نقش‌محور و حفاظت از فایل و رویداد	نشست، CSRF، توابع مجوز دامنه، کنترل داندلود و احراز هویت WebSocket به‌صورت چندلایه اجرا می‌شوند و مجوز عضویت هنگام رویداد دوباره بررسی می‌گردد.

جنبه طراحی	انتظار معماری	تحقق در محصول نهایی
استقرار	محیط اجرای تکرارپذیر و سرویس‌های قابل جداسازی	Docker Compose اجزای رابط، API، پایگاه داده، Redis، MinIO و Celery را مطابق وابستگی‌های معماری بالا می‌آورد و تنظیمات از environment تأمین می‌شوند.